CS5275 – The Algorithm Designer's Toolkit (S2 AY2025/26)

Lecture 11:

Boolean functions – BLR linearity test

The finite field of order 2

In this lecture, we focus on Boolean functions

$$f : \mathbb{F}_2^n \to \mathbb{R},$$

where $\mathbb{F}_2 = \{0,1\}$ is a the finite field of order 2.

The finite field of order 2

In this lecture, we focus on Boolean functions

$$f : \mathbb{F}_2^n \to \mathbb{R},$$

where $\mathbb{F}_2 = \{0,1\}$ is a the finite field of order 2.

Multiplication is modular 2 (**AND**):

- $0 \cdot 0 = 0$
- $0 \cdot 1 = 0$
- $1 \cdot 0 = 0$
- $1 \cdot 1 = 1$

Addition is modular 2 (**XOR / parity**):

- $0 + 0 = 0$
- $0 + 1 = 1$
- $1 + 0 = 1$
- $1 + 1 = 0$

The finite field of order 2

In this lecture, we focus on Boolean functions

$$f : \mathbb{F}_2^n \to \mathbb{R},$$

where $\mathbb{F}_2 = \{0,1\}$ is a the finite field of order 2.

Multiplication is modular 2 (**AND**):

- $0 \cdot 0 = 0$
- $0 \cdot 1 = 0$
- $1 \cdot 0 = 0$
- $1 \cdot 1 = 1$

Addition is modular 2 (**XOR / parity**):

- $0 + 0 = 0$
- $0 + 1 = 1$
- $1 + 0 = 1$
- $1 + 1 = 0$

How do we use the tools developed for functions of the form

$$f : \{-1,1\}^n \to \mathbb{R}\,?$$

Consider the mapping:

- Viewing $0 \in \mathbb{F}_2$ as 1
- Viewing $1 \in \mathbb{F}_2$ as -1

Linear functions

A function $f : \mathbb{F}_2^n \to \mathbb{F}_2$ is **linear**.

$\updownarrow$

$f(x) = \sum_{i \in [n]} c_i x_i$ for some coefficients $c_i \in \mathbb{F}_2$.

$\updownarrow$

$f(x) = \sum_{i \in S} x_i$ for some $S \subseteq [n]$.

Linear functions ↔ parity functions

A function $f : \mathbb{F}_2^n \to \mathbb{F}_2$ is **linear**.

$f(x) = \sum_{i \in [n]} c_i x_i$ for some coefficients $c_i \in \mathbb{F}_2$.

$f(x) = \sum_{i \in S} x_i$ for some $S \subseteq [n]$.

$f(x) = x^S = \chi_S$ for some $S \subseteq [n]$.

Consider the mapping:

- Viewing $0 \in \mathbb{F}_2$ as 1
- Viewing $1 \in \mathbb{F}_2$ as -1

Distances

$$\mathrm{dist}(f, g) = \Pr[f \neq g]$$

$=$ fraction of inputs on which f and g disagrees.

- How do we measure the distance between two Boolean functions?

If we focus on functions $\mathbb{F}_2^n \to \mathbb{F}_2$ and viewing them as vectors in $\mathbb{F}_2^{2^n}$, then $\mathrm{dist}(f, g)$ equals $2^n \cdot$ Hamming distance.

Distances

$$\mathrm{dist}(f,g) = \Pr[f \neq g]$$

$=$ fraction of inputs on which f and g disagrees.

- How do we measure the distance between two Boolean functions?

If we focus on functions $\mathbb{F}_2^n \to \mathbb{F}_2$ and viewing them as vectors in $\mathbb{F}_2^{2^n}$, then $\mathrm{dist}(f,g)$ equals $2^n \cdot$ Hamming distance.

Observation: For any Boolean functions f and $g : \{-1,1\}^n \to \{-1,1\}$,

$$\langle f,g\rangle = \mathbb{E}[fg] = \Pr[f = g] - \Pr[f \neq g] = 1 - 2\mathrm{dist}(f,g).$$

Distances

$\mathrm{dist}(f,g) = \Pr[f \neq g]$ = fraction of inputs on which f and g disagrees.

- How do we measure the distance between two Boolean functions?

Intuition: Fourier coefficients capture distances to linear functions.

Corollary: $\hat{f}(S) = \langle f, \chi_S \rangle = 1 - 2\mathrm{dist}(f, \chi_S)$.

If we focus on functions $\mathbb{F}_2^n \to \mathbb{F}_2$ and viewing them as vectors in $\mathbb{F}_2^{2^n}$, then $\mathrm{dist}(f,g)$ equals $2^n \cdot$ Hamming distance.

Observation: For any Boolean functions f and $g : \{-1,1\}^n \to \{-1,1\}$,

$$\langle f, g \rangle = \mathbb{E}[fg] = \Pr[f = g] - \Pr[f \neq g] = 1 - 2\mathrm{dist}(f,g).$$

Intuition: Fourier analysis is often useful when a problem involves distances.

Distance to $\mathcal{P}$

$$\mathrm{dist}(f, g) = \Pr[f \neq g]$$

$$\mathrm{dist}(f, \mathcal{P}) = \min_{g \in \mathcal{P}} \Pr[f \neq g]$$

- A property is a set of functions $\mathcal{P}$.

  For example, the collection of linear functions is a property.

- f is ϵ-**close** to $\mathcal{P}$ if $\mathrm{dist}(f, \mathcal{P}) \leq \epsilon$.
- f is ϵ-**far** from $\mathcal{P}$ if $\mathrm{dist}(f, \mathcal{P}) > \epsilon$.

Property testing

$$\mathrm{dist}(f, g) = \Pr[f \neq g]$$

$$\mathrm{dist}(f, \mathcal{P}) = \min_{g \in \mathcal{P}} \Pr[f \neq g]$$

- A property is a set of functions $\mathcal{P}$.

- f is ϵ-**close** to $\mathcal{P}$ if $\mathrm{dist}(f, \mathcal{P}) \leq \epsilon$.
- f is ϵ-**far** from $\mathcal{P}$ if $\mathrm{dist}(f, \mathcal{P}) > \epsilon$.

Property testing for $\mathcal{P}$:
- **Input:** a function f.
- **Output:** accept or reject.

Requirement:
- **Soundness:** If f is ϵ-far from $\mathcal{P}$, then the algorithm rejects f.
- **Completeness:** If $f \in \mathcal{P}$, then the algorithm accepts f.

Due to the ϵ-slack, this task can often be accomplished in sublinear time.

Linearity testing

Our focus: $\mathcal{P}$ is the collection of all linear functions : $\mathbb{F}_2^n \to \mathbb{F}_2$.

Property testing for $\mathcal{P}$:
- **Input:** a function f.
- **Output:** accept or reject.

Requirement:
- **Soundness:** If f is ϵ-far from $\mathcal{P}$, then the algorithm rejects f.
- **Completeness:** If $f \in \mathcal{P}$, then the algorithm accepts f.

Due to the ϵ-slack, this task can often be accomplished in sublinear time.

Linearity testing

Next: a simple algorithm that uses only 3 queries of f:

- If f is ϵ-far from $\mathcal{P}$, then f is rejected with probability at least ϵ.
- If $f \in \mathcal{P}$, then f is accepted with probability 1.

Our focus: $\mathcal{P}$ is the collection of all linear functions : $\mathbb{F}_2^n \to \mathbb{F}_2$.

Property testing for $\mathcal{P}$:

- **Input:** a function f.
- **Output:** accept or reject.

Requirement:

- **Soundness:** If f is ϵ-far from $\mathcal{P}$, then the algorithm rejects f.
- **Completeness:** If $f \in \mathcal{P}$, then the algorithm accepts f.

Due to the ϵ-slack, this task can often be accomplished in sublinear time.

Linearity testing

Can amplify the success probability to $1-\delta$ by repeating for $O\left(\frac{\log\frac{1}{\delta}}{\epsilon}\right)$ times.

Next: a simple algorithm that uses only 3 queries of f:

- If f is ϵ-far from $\mathcal{P}$, then f is rejected with probability at least ϵ.
- If $f \in \mathcal{P}$, then f is accepted with probability 1.

Our focus: $\mathcal{P}$ is the collection of all linear functions : $\mathbb{F}_2^n \to \mathbb{F}_2$.

Property testing for $\mathcal{P}$:

- **Input:** a function f.
- **Output:** accept or reject.

Requirement:

- **Soundness:** If f is ϵ-far from $\mathcal{P}$, then the algorithm rejects f.
- **Completeness:** If $f \in \mathcal{P}$, then the algorithm accepts f.

Due to the ϵ-slack, this task can often be accomplished in sublinear time.

Blum–Luby–Rubinfeld linearity test

BLR test:

- Choose $\boldsymbol{x} \sim \mathbb{F}_2^n$ and $\boldsymbol{y} \sim \mathbb{F}_2^n$ independently.
- Query f at $\boldsymbol{x}$, $\boldsymbol{y}$, and $\boldsymbol{x} + \boldsymbol{y}$.
- Accept if $f(\boldsymbol{x}) + f(\boldsymbol{y}) = f(\boldsymbol{x} + \boldsymbol{y})$.

Blum–Luby–Rubinfeld linearity test

BLR test:

- Choose $\boldsymbol{x} \sim \mathbb{F}_2^n$ and $\boldsymbol{y} \sim \mathbb{F}_2^n$ independently.
- Query f at $\boldsymbol{x}$, $\boldsymbol{y}$, and $\boldsymbol{x} + \boldsymbol{y}$.
- Accept if $f(\boldsymbol{x}) + f(\boldsymbol{y}) = f(\boldsymbol{x} + \boldsymbol{y})$.

Completeness: If $f \in \mathcal{P}$, then the algorithm accepts f with probability 1.

If f is linear, then $f(x) + f(y) = f(x + y)$ for any x and y.

Blum–Luby–Rubinfeld linearity test

BLR test:
- Choose $\boldsymbol{x} \sim \mathbb{F}_2^n$ and $\boldsymbol{y} \sim \mathbb{F}_2^n$ independently.
- Query f at $\boldsymbol{x}$, $\boldsymbol{y}$, and $\boldsymbol{x}+\boldsymbol{y}$.
- Accept if $f(\boldsymbol{x}) + f(\boldsymbol{y}) = f(\boldsymbol{x}+\boldsymbol{y})$.

Completeness: If $f \in \mathcal{P}$, then the algorithm accepts f with probability 1.

Soundness: If f is ϵ-far from $\mathcal{P}$, then the algorithm rejects f with probability at least ϵ.

Addition becomes multiplication

We view f as a function $\mathbb{F}_2^n \to \{-1,1\}$:
- Viewing $0 \in \mathbb{F}_2$ as 1
- Viewing $1 \in \mathbb{F}_2$ as -1

Thus, we can apply Fourier analysis.

Algorithm accepts f. $\longleftrightarrow$ $f(\boldsymbol{x})f(\boldsymbol{y}) = f(\boldsymbol{x}+\boldsymbol{y})$ $\longleftrightarrow$ $f(\boldsymbol{x})f(\boldsymbol{y})f(\boldsymbol{x}+\boldsymbol{y}) = 1$

Blum–Luby–Rubinfeld linearity test

$$\Pr[\text{algorithm accepts } f]$$
$$= \mathbb{E}_{x,y}\left[\frac{1}{2} + \frac{1}{2} f(x) f(y) f(x+y)\right]$$
$$= \frac{1}{2} + \frac{1}{2}\mathbb{E}_x\left[f(x)\mathbb{E}_y[f(y)f(x+y)]\right]$$

Algorithm accepts f. ⟷ $f(x)f(y) = f(x+y)$ ⟷ $f(x)f(y)f(x+y) = 1$

Blum–Luby–Rubinfeld linearity test

$$\Pr[\text{algorithm accepts } f]$$
$$= \mathbb{E}_{x,y}\left[\frac{1}{2} + \frac{1}{2} f(\boldsymbol{x}) f(\boldsymbol{y}) f(\boldsymbol{x}+\boldsymbol{y})\right]$$
$$= \frac{1}{2} + \frac{1}{2}\mathbb{E}_{x}\left[f(\boldsymbol{x})\mathbb{E}_{y}[f(\boldsymbol{y}) f(\boldsymbol{x}+\boldsymbol{y})]\right]$$
$$= \frac{1}{2} + \frac{1}{2}\mathbb{E}_{x}[f(\boldsymbol{x}) \cdot (f * f)(\boldsymbol{x})]$$

For any two functions $f, g : \mathbb{F}_2^n \to \mathbb{R}$, their **convolution** is $(f * g)(x) = \mathbb{E}_{\boldsymbol{y}}[f(\boldsymbol{y}) g(x+\boldsymbol{y})] = \mathbb{E}_{\boldsymbol{y}}[f(\boldsymbol{y}) g(x-\boldsymbol{y})]$.

Blum–Luby–Rubinfeld linearity test

$$\Pr[\text{algorithm accepts } f]$$
$$= \mathbb{E}_{x,y}\left[\frac{1}{2} + \frac{1}{2} f(\boldsymbol{x}) f(\boldsymbol{y}) f(\boldsymbol{x}+\boldsymbol{y})\right]$$
$$= \frac{1}{2} + \frac{1}{2}\mathbb{E}_{x}\left[f(\boldsymbol{x})\mathbb{E}_{y}[f(\boldsymbol{y})f(\boldsymbol{x}+\boldsymbol{y})]\right]$$
$$= \frac{1}{2} + \frac{1}{2}\mathbb{E}_{x}[f(\boldsymbol{x}) \cdot (f * f)(\boldsymbol{x})]$$
$$= \frac{1}{2} + \frac{1}{2}\sum_{S\subseteq[n]} \hat{f}(S) \cdot \widehat{f * f}(S)$$
$$= \frac{1}{2} + \frac{1}{2}\sum_{S\subseteq[n]} \hat{f}(S)^3$$

For any two functions $f, g : \mathbb{F}_2^n \to \mathbb{R}$, their **convolution** is $(f * g)(x) = \mathbb{E}_{\boldsymbol{y}}[f(\boldsymbol{y})g(x+\boldsymbol{y})] = \mathbb{E}_{\boldsymbol{y}}[f(\boldsymbol{y})g(x-\boldsymbol{y})]$.

Plancherel's theorem: $\mathbb{E}[fg] = \sum_{S\subseteq[n]} \hat{f}(S)\hat{g}(S)$

Lemma: $\widehat{f * g}(S) = \hat{f}(S)\hat{g}(S)$

Blum–Luby–Rubinfeld linearity test

$$
\begin{aligned}
&\Pr[\text{algorithm accepts } f] \\
&= \mathbb{E}_{x,y}\left[\frac{1}{2} + \frac{1}{2} f(x) f(y) f(x+y)\right] \\
&= \frac{1}{2} + \frac{1}{2}\mathbb{E}_x\left[f(x)\mathbb{E}_y[f(y)f(x+y)]\right] \\
&= \frac{1}{2} + \frac{1}{2}\mathbb{E}_x[f(x)\cdot(f*f)(x)] \\
&= \frac{1}{2} + \frac{1}{2}\sum_{S\subseteq[n]} \hat{f}(S)\cdot\widehat{f*f}(S) \\
&= \frac{1}{2} + \frac{1}{2}\sum_{S\subseteq[n]} \hat{f}(S)^3 \\
&\le \frac{1}{2} + \frac{1}{2}\max_{S\subseteq[n]} \hat{f}(S) \\
&\le \frac{1}{2} + \frac{1}{2}\max_{S\subseteq[n]} \left(1 - 2\mathrm{dist}(f,\chi_S)\right)
\end{aligned}
$$

Since the range of the output of f is $\{-1,1\}$,

$$\sum_{S\subseteq[n]} \hat{f}(S)^2 = 1$$

$$\hat{f}(S) = 1 - 2\mathrm{dist}(f,\chi_S)$$

Blum–Luby–Rubinfeld linearity test

$$
\begin{aligned}
&\Pr[\text{algorithm accepts } f] \\
&= \mathbb{E}_{x,y}\left[\frac{1}{2} + \frac{1}{2} f(x) f(y) f(x+y)\right] \\
&= \frac{1}{2} + \frac{1}{2}\mathbb{E}_x\left[f(x)\mathbb{E}_y[f(y)f(x+y)]\right] \\
&= \frac{1}{2} + \frac{1}{2}\mathbb{E}_x[f(x)\cdot(f*f)(x)] \\
&= \frac{1}{2} + \frac{1}{2}\sum_{S\subseteq[n]} \hat{f}(S)\cdot\widehat{f*f}(S) \\
&= \frac{1}{2} + \frac{1}{2}\sum_{S\subseteq[n]} \hat{f}(S)^3 \\
&\leq \frac{1}{2} + \frac{1}{2}\max_{S\subseteq[n]} \hat{f}(S) \\
&\leq \frac{1}{2} + \frac{1}{2}\max_{S\subseteq[n]}\left(1 - 2\mathrm{dist}(f,\chi_S)\right) \\
&< 1-\epsilon
\end{aligned}
$$

f is ϵ-far from being linear $\rightarrow \mathrm{dist}(f,\chi_S) > \epsilon$

Soundness: If f is ϵ-far from $\mathcal{P}$, then the algorithm rejects f with probability at least ϵ.

Blum–Luby–Rubinfeld linearity test

- **Summary:**
  - BLR test decides whether f is close to some linear function χ_S.
  - However, the test does not reveal the function χ_S.

Exercise: n queries are necessary and sufficient to determine χ_S.

Local correction

- **Summary:**
  - BLR test decides whether f is close to some linear function χ_S.
  - However, the test does not reveal the function χ_S.

Exercise: n queries are necessary and sufficient to determine χ_S.

Local correctability:

- Suppose $f : \mathbb{F}_2^n \to \{-1,1\}$ is ϵ-close to χ_S for some unknown S.
- For any input $x \in \mathbb{F}_2^n$:
  - $\chi_S(x)$ can be determined using 3 queries to f with probability at least $1 - 2\epsilon$.

Local correction

Local correctability:

- Suppose $f : \mathbb{F}_2^n \to \{-1,1\}$ is ϵ-close to χ_S for some unknown S.
- For any input $x \in \mathbb{F}_2^n$:
  - $\chi_S(x)$ can be determined using 3 queries to f with probability at least $1 - 2\epsilon$.

- Choose $\boldsymbol{y} \sim \mathbb{F}_2^n$.
- Query f at $\boldsymbol{y}$, and $x + \boldsymbol{y}$.
- Output $f(\boldsymbol{y})f(x + \boldsymbol{y})$.

Local correction

$$\Pr[f(\boldsymbol{y}) \neq \chi_S(\boldsymbol{y})] \leq \epsilon$$

$$\Pr[f(x+\boldsymbol{y}) \neq \chi_S(x+\boldsymbol{y})] \leq \epsilon$$

$\Pr[f \neq \chi_S] \leq \epsilon$

Both $\boldsymbol{y}$ and $x + \boldsymbol{y}$ are uniformly distributed on $\mathbb{F}_2^n$.

Local correctability:

- Suppose $f : \mathbb{F}_2^n \to \{-1,1\}$ is ϵ-close to χ_S for some unknown S.
- For any input $x \in \mathbb{F}_2^n$:
  - $\chi_S(x)$ can be determined using 3 queries to f with probability at least $1 - 2\epsilon$.

- Choose $\boldsymbol{y} \sim \mathbb{F}_2^n$.
- Query f at $\boldsymbol{y}$, and $x + \boldsymbol{y}$.
- Output $f(\boldsymbol{y})f(x + \boldsymbol{y})$.

Local correction

$$f(\boldsymbol{y})f(x+\boldsymbol{y}) = \chi_S(\boldsymbol{y})\chi_S(x+\boldsymbol{y}) = \chi_S(x)$$

conditioning on these bad events not happening

$$\Pr[f(\boldsymbol{y}) \neq \chi_S(\boldsymbol{y})] \leq \epsilon$$

$$\Pr[f(x+\boldsymbol{y}) \neq \chi_S(x+\boldsymbol{y})] \leq \epsilon$$

$\Pr[f \neq \chi_S] \leq \epsilon$

Both $\boldsymbol{y}$ and $x+\boldsymbol{y}$ are uniformly distributed on $\mathbb{F}_2^n$.

Local correctability:

- Suppose $f : \mathbb{F}_2^n \to \{-1,1\}$ is ϵ-close to χ_S for some unknown S.
- For any input $x \in \mathbb{F}_2^n$:
  - $\chi_S(x)$ can be determined using 3 queries to f with probability at least $1 - 2\epsilon$.

- Choose $\boldsymbol{y} \sim \mathbb{F}_2^n$.
- Query f at $\boldsymbol{y}$, and $x+\boldsymbol{y}$.
- Output $f(\boldsymbol{y})f(x+\boldsymbol{y})$.

Fourier analysis of convolution

- To finish the proof, it remains to prove the lemma:

$$\widehat{f * g}(S) = \hat{f}(S)\hat{g}(S)$$

For any two functions $f, g : \mathbb{F}_2^n \to \mathbb{R}$, their **convolution** is $(f * g)(x) = \mathbb{E}_{\boldsymbol{y}}[f(\boldsymbol{y})g(x + \boldsymbol{y})] = \mathbb{E}_{\boldsymbol{y}}[f(\boldsymbol{y})g(x - \boldsymbol{y})]$.

Fourier analysis of convolution

- To finish the proof, it remains to prove the lemma:

$$\widehat{f * g}(S) = \hat{f}(S)\hat{g}(S)$$

$$\widehat{f * g}(S) = \langle f * g, \chi_S \rangle = \mathbb{E}_{\boldsymbol{x} \sim \mathbb{F}_2^n}[(f * g)(\boldsymbol{x}) \cdot \chi_S(\boldsymbol{x})] = \mathbb{E}_{\boldsymbol{x} \sim \mathbb{F}_2^n}\left[\mathbb{E}_{\boldsymbol{y} \sim \mathbb{F}_2^n}[f(\boldsymbol{y})g(\boldsymbol{x} - \boldsymbol{y})] \cdot \chi_S(\boldsymbol{x})\right]$$

Recall: $\hat{f}(S) = \langle f, \chi_S \rangle = \mathbb{E}[f(\boldsymbol{x})\chi_S(\boldsymbol{x})]$

For any two functions $f, g : \mathbb{F}_2^n \to \mathbb{R}$, their **convolution** is $(f * g)(x) = \mathbb{E}_{\boldsymbol{y}}[f(\boldsymbol{y})g(x + \boldsymbol{y})] = \mathbb{E}_{\boldsymbol{y}}[f(\boldsymbol{y})g(x - \boldsymbol{y})]$.

Fourier analysis of convolution

- To finish the proof, it remains to prove the lemma:

$$\widehat{f * g}(S) = \hat{f}(S)\hat{g}(S)$$

$$\widehat{f * g}(S) = \langle f * g, \chi_S \rangle = \mathbb{E}_{\boldsymbol{x} \sim \mathbb{F}_2^n}[(f * g)(\boldsymbol{x}) \cdot \chi_S(\boldsymbol{x})] = \mathbb{E}_{\boldsymbol{x} \sim \mathbb{F}_2^n}\left[\mathbb{E}_{\boldsymbol{y} \sim \mathbb{F}_2^n}[f(\boldsymbol{y})g(\boldsymbol{x} - \boldsymbol{y})] \cdot \chi_S(\boldsymbol{x})\right]$$

$$= \mathbb{E}_{\boldsymbol{y} \sim \mathbb{F}_2^n,\, \boldsymbol{z} \sim \mathbb{F}_2^n}[f(\boldsymbol{y})g(\boldsymbol{z}) \cdot \chi_S(\boldsymbol{y} + \boldsymbol{z})] \qquad \boldsymbol{z = x - y}$$

$$= \mathbb{E}_{\boldsymbol{y} \sim \mathbb{F}_2^n,\, \boldsymbol{z} \sim \mathbb{F}_2^n}[f(\boldsymbol{y})\chi_S(\boldsymbol{y}) \cdot g(\boldsymbol{z})\chi_S(\boldsymbol{z})] \qquad \chi_S(\boldsymbol{y})\chi_S(\boldsymbol{z}) = \chi_S(\boldsymbol{y} + \boldsymbol{z})$$

For any two functions $f, g : \mathbb{F}_2^n \to \mathbb{R}$, their **convolution** is $(f * g)(x) = \mathbb{E}_{\boldsymbol{y}}[f(\boldsymbol{y})g(x + \boldsymbol{y})] = \mathbb{E}_{\boldsymbol{y}}[f(\boldsymbol{y})g(x - \boldsymbol{y})]$.

Fourier analysis of convolution

- To finish the proof, it remains to prove the lemma:

$$\widehat{f * g}(S) = \hat{f}(S)\hat{g}(S)$$

$$\widehat{f * g}(S) = \langle f * g, \chi_S \rangle = \mathbb{E}_{\boldsymbol{x}\sim\mathbb{F}_2^n}[(f * g)(\boldsymbol{x}) \cdot \chi_S(\boldsymbol{x})] = \mathbb{E}_{\boldsymbol{x}\sim\mathbb{F}_2^n}\left[\mathbb{E}_{\boldsymbol{y}\sim\mathbb{F}_2^n}[f(\boldsymbol{y})g(\boldsymbol{x}-\boldsymbol{y})] \cdot \chi_S(\boldsymbol{x})\right]$$

$$= \mathbb{E}_{\boldsymbol{y}\sim\mathbb{F}_2^n,\, \boldsymbol{z}\sim\mathbb{F}_2^n}[f(\boldsymbol{y})g(\boldsymbol{z}) \cdot \chi_S(\boldsymbol{y}+\boldsymbol{z})] \qquad \boldsymbol{z} = \boldsymbol{x} - \boldsymbol{y}$$

$$= \mathbb{E}_{\boldsymbol{y}\sim\mathbb{F}_2^n,\, \boldsymbol{z}\sim\mathbb{F}_2^n}[f(\boldsymbol{y})\chi_S(\boldsymbol{y}) \cdot g(\boldsymbol{z})\chi_S(\boldsymbol{z})] \qquad \chi_S(\boldsymbol{y})\chi_S(\boldsymbol{z}) = \chi_S(\boldsymbol{y}+\boldsymbol{z})$$

$$= \mathbb{E}_{\boldsymbol{y}\sim\mathbb{F}_2^n,}[f(\boldsymbol{y})\chi_S(\boldsymbol{y})] \cdot \mathbb{E}_{\boldsymbol{z}\sim\mathbb{F}_2^n}[g(\boldsymbol{z})\chi_S(\boldsymbol{z})]$$

$$= \hat{f}(S)\hat{g}(S)$$

For any two functions $f, g : \mathbb{F}_2^n \to \mathbb{R}$, their **convolution** is $(f * g)(x) = \mathbb{E}_{\boldsymbol{y}}[f(\boldsymbol{y})g(x+\boldsymbol{y})] = \mathbb{E}_{\boldsymbol{y}}[f(\boldsymbol{y})g(x-\boldsymbol{y})]$.

Motivation

- BLR test was originally designed for **program checkers** and **self-correctors**.

BLR test allows us to quickly check whether a program correctly computes a linear function without running it on every input.

BLR test offers a mechanism that can recover the correct output even if the program is slightly faulty.

Further developments

- The BLR test introduced a **powerful paradigm**:

> Global properties can be verified using only a few random local checks.

- Instead of examining an entire object:
  - Randomly sample a small number of positions.
  - Check whether local consistency conditions hold.
  - Conclude whether the object is likely correct or far from correct

- This idea influenced several areas in theoretical computer science.

Further developments

Locally testable code:

- In coding theory, a codeword represents encoded data.
- A locally testable code allows us to check whether a string is:
  - a valid codeword, or
  - far from any valid codeword

  by reading only a few positions of the string.

Further developments

Locally testable code:

- In coding theory, a codeword represents encoded data.
- A locally testable code allows us to check whether a string is:
  - a valid codeword, or
  - far from any valid codeword

  by reading only a few positions of the string.

Probabilistically checkable proof (PCP):

- A probabilistically checkable proof allows a verifier to check the correctness of a proof by reading only a few randomly chosen bits.

PCP theorem

$$\mathrm{NP} = \mathrm{PCP}\big(O(\log n), O(1)\big)$$

A problem is in NP if there exists a polynomial-time verification algorithm such that:

- **Completeness:** For every yes-instance, there exists a certificate that the verifier accepts.
- **Soundness:** For every no-instance, all certificates are rejected by the verifier.

A problem is in $\mathrm{PCP}(r, s)$ if there exists a polynomial-time verification algorithm that uses r random bits to query s bits of the certificate such that:

- **Completeness:** For every yes-instance, there exists a certificate that the verifier accepts with probability 1.
- **Soundness:** For every no-instance, all certificates are rejected by the verifier with probability at least $1/2$.

PCP theorem

$$\mathrm{NP} = \mathrm{PCP}\big(O(\log n), O(1)\big)$$

Many applications to hardness of approximation

Exercise: There exists a constant $\epsilon > 0$ such that it is **NP-hard** to distinguish between:

- Instances of 3SAT that are satisfiable.
- instances of 3SAT in which at most a $(1 - \epsilon)$-fraction of clauses can be satisfied.

Outlook

- **Next:** Applying Fourier analysis to the analysis of voting rules.

+ impossibility results

also known as social choice functions

References

- Sections 1.5–1.6 of https://arxiv.org/abs/2105.10386